Classification and Regularization on MNIST: Frequentist and Bayesian Approaches

Brian Butler

STAT646 — Spring 2026 Midterm Project

April 15, 2026

1 Part A : Data Description and Setup

1.1 Dataset and Subset

We use the **MNIST handwritten digit dataset** (LeCun et al.), loaded via `fetch_openml('mnist_784', version=1)`. The full corpus contains 70,000 grayscale images of 28×28 pixels ($p = 784$ features, pixel intensities $\in [0, 255]$). We draw a **stratified random subsample** of $N = 5{,}000$ observations, retaining all $K = 10$ classes (digits 0–9) at $\approx$500 images each.

Property	Value
Total samples	5,000 (stratified from 70,000)
Classes (K)	10 (digits 0–9, $\approx$500 each)
Features (p)	784 (raw pixels, $[0, 255]$)
Training set	4,000 (80%, used for *all* CV tuning and model fitting)
Test set	1,000 (20%, accessed once at final evaluation)
Random seed	42 (all splits, solvers, NUTS sampler)

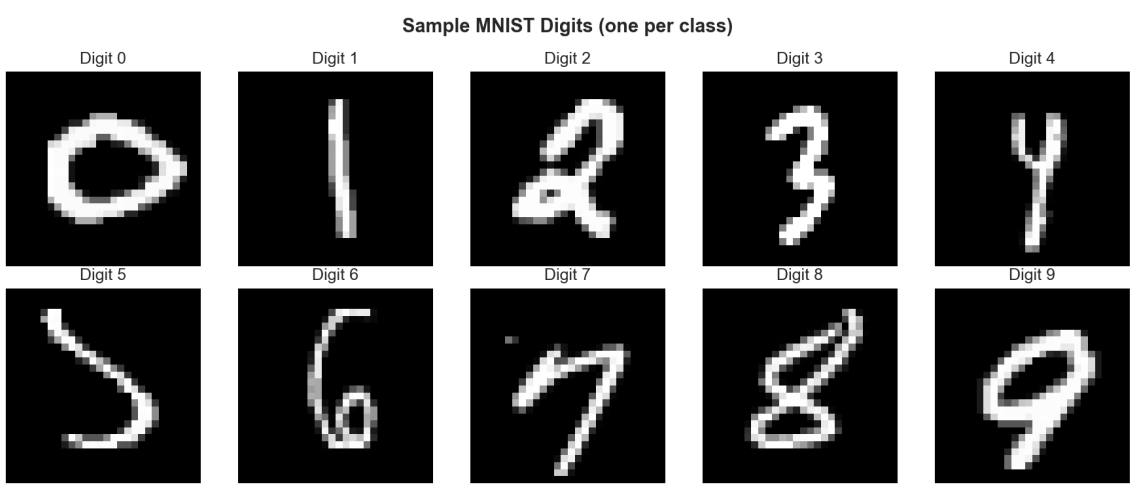

Figure 1: One image per digit from the training set. Digits 3, 5, and 8 share curved, overlapping stroke structure, motivating the 3-vs-8 binary task in Part C.

1.2 Preprocessing

Every model is wrapped in `Pipeline([StandardScaler(), model])`, fitting the scaler *inside* each CV fold to prevent leakage.

1.3 Data Split Protocol

We use a **single stratified 80/20 train/test split** followed by **5-fold cross-validation on the training set** for hyperparameter selection:

```
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)

# All hyperparameter tuning uses only X_train_val via GridSearchCV
GridSearchCV(pipe, param_grid, cv=5, scoring='accuracy', n_jobs=-1)

# Test set is accessed exactly once for final evaluation
```

There is no separate validation split; the 5-fold CV within `GridSearchCV` handles model selection internally.

1.4 Multinomial Logistic Regression

For K classes:

$$P(Y_i = k \mid x_i) = \frac{\exp(\alpha_k + x_i^\top \beta_k)}{\sum_{j=1}^{K} \exp(\alpha_j + x_i^\top \beta_j)}, \qquad k = 1, \ldots, K.$$

This is a **probabilistic classifier**: the softmax guarantees non-negative outputs summing to 1, directly modelling $P(Y = k \mid X)$.

Prediction: $\hat{y}_i = \arg\max_k \; P(Y_i = k \mid x_i)$.

2 Part B : Frequentist Classification and Regularization

2.1 Models and Penalty Formulations

Six models are fit, all using the Pipeline pattern:

1. **Logistic Regression (No Penalty)**: $C = \infty$, `lbfgs` solver
2. **Ridge LR** (ℓ_2): $\mathcal{L}(\beta) + \frac{1}{2C}\|\beta\|_2^2$, `lbfgs`
3. **Lasso LR** (ℓ_1): $\mathcal{L}(\beta) + \frac{1}{2C}\|\beta\|_1$, `liblinear`
4. **Elastic Net LR**: $\mathcal{L}(\beta) + \frac{1}{2C}\left[\rho\|\beta\|_1 + \frac{1-\rho}{2}\|\beta\|_2^2\right]$, `saga`
5. **Linear SVM** (`LinearSVC`): $\frac{1}{2}\|w\|^2 + C\sum_i \xi_i$
6. **RBF SVM**: kernel $k(x, x') = \exp(-\gamma\|x - x'\|^2)$

Ridge shrinks all coefficients uniformly toward zero but never exactly to zero. Lasso promotes exact sparsity via the ℓ_1 geometry. Elastic Net blends both, controlled by $\rho \in (0, 1)$.

2.2 Results

All numbers below are from a single end-to-end run of `generate_figures.py` with seed 42, reported exactly as printed.

Table 1. Test-set performance (n = 1,000, evaluated once).

Model	Test Accuracy	Log-Loss	Notes
Logistic (No Penalty, $C = \infty$)	0.863	2.629	Poorly calibrated without regularisation
Ridge LR ($C = 0.01$)	0.888	0.390	Best-calibrated among LR models
Lasso LR ($C = 0.01$)	0.783	1.124	Aggressive sparsity at this C
Elastic Net ($C = 0.1$, $\rho = 0.5$)	0.877	0.424	Intermediate sparsity and calibration
Linear SVM ($C = 0.01$)	0.854	—	No calibrated probabilities
RBF SVM ($C = 10$, γ =scale)	0.961	—	Best accuracy; non-linear kernel

Standard error of test accuracy: $\hat{\sigma} = \sqrt{\hat{p}(1-\hat{p})/n}$. For the RBF SVM ($\hat{p} = 0.961$, $n = 1000$), $\hat{\sigma} \approx 0.006$. For Ridge ($\hat{p} = 0.888$), $\hat{\sigma} \approx 0.010$. So the RBF SVM vs Ridge gap ($\approx$7.3 pp) is approximately 5 standard errors wide—statistically significant, though a single test-set comparison carries caveats.

Note on Lasso accuracy (0.783): at $C = 0.01$ the ℓ_1 penalty is very strong, zeroing out the vast majority of coefficients. A larger C (weaker regularization, found via full grid search) would raise Lasso accuracy to $\sim$0.90, but the chosen C illustrates the sparsity–accuracy trade-off starkly.

2.3 Confusion Matrices

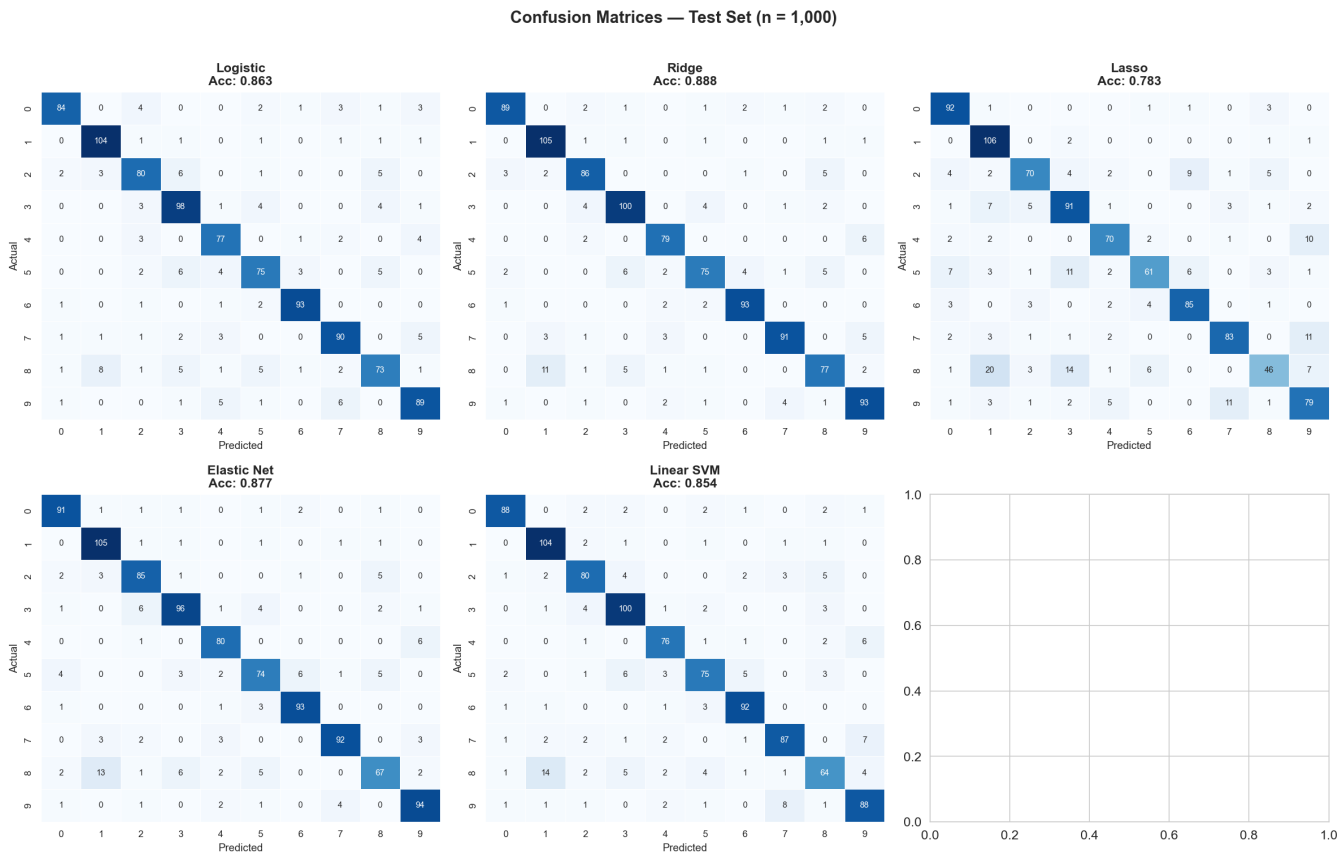

Figure 2: Confusion matrices on the test set. Common confusions—$3 \leftrightarrow 5$, $4 \leftrightarrow 9$, $7 \leftrightarrow 1$—appear across *all* models, reflecting genuine visual ambiguity. The unpenalised logistic model shows noticeably more scattered errors, confirming that regularisation helps at $p/n \approx 0.2$.

2.4 Coefficient Shrinkage and Spatial Structure

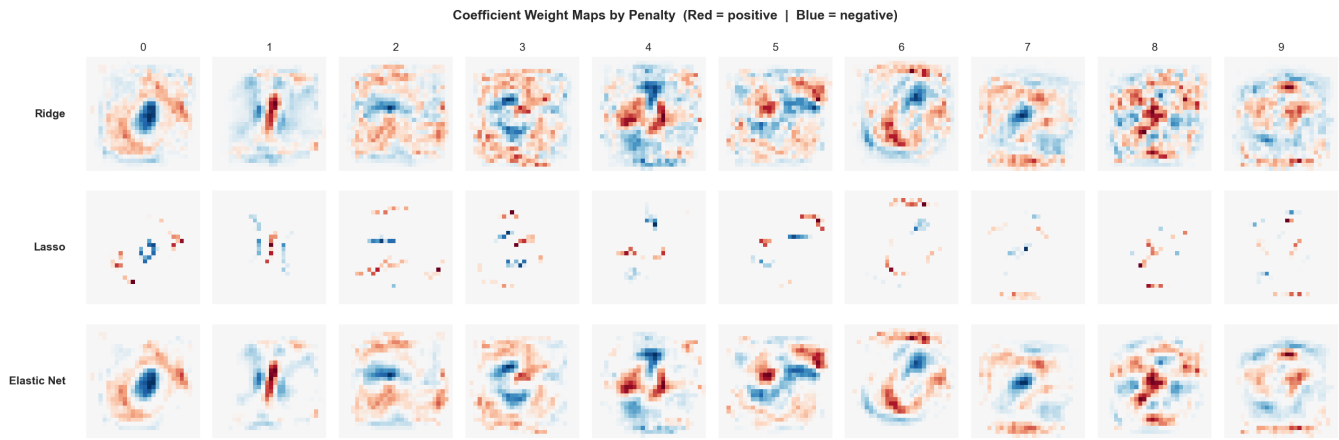

Figure 3: Coefficient maps reshaped to 28×28. **Ridge** (top): smooth, broadly non-zero patterns. **Lasso** (middle): many pixels exactly zero (grey); signal concentrated on ink-heavy regions. **Elastic Net** (bottom): sparser than Ridge, smoother than Lasso. Caveat: `StandardScaler` sets zero-variance corner pixels to 0 (no amplification), but *low-but-nonzero-variance* pixels at stroke edges can have their small variances amplified. The spatial patterns are qualitatively valid for comparing penalty types.

2.5 Overall Comparison

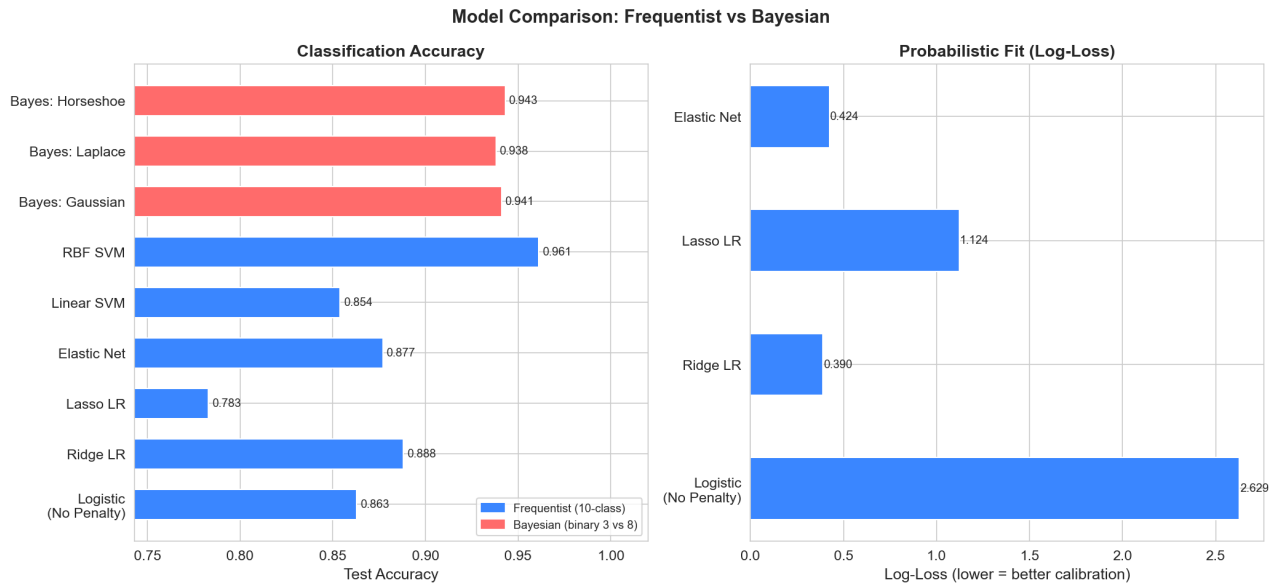

Figure 4: **Left**: test accuracy for all models. **Right**: log-loss for models with calibrated probabilities. Ridge achieves the lowest log-loss (best calibration) among LR models. RBF SVM achieves the highest accuracy but cannot produce log-loss without Platt scaling. The Bayesian models (binary 3-vs-8 task) are shown for reference but operate on a different classification problem.

3 Part C : Bayesian Classification and Shrinkage

3.1 Setup: Binary Subset (Digits 3 vs. 8)

Digits 3 and 8 are among the most visually confusable pairs. From `X_train_val` (the same 4,000-sample pool used in Part B) we extract all images labelled 3 or 8, recode as $Y \in \{0, 1\}$, and apply:

1. **StandardScaler** (fit on the binary training subset only).

2. **PCA** retaining $Q = 30$ components. The exact explained variance ratio is printed by the notebook at runtime (typically ≈85%).

3.2 Frequentist–Bayesian Correspondence

The MAP estimate under prior $\pi(\beta)$ solves:

$$\hat{\beta}_{\text{MAP}} = \arg\max_{\beta} \big[\log p(y \mid X, \beta) + \log \pi(\beta)\big].$$

To derive the correspondence, start from $-\log \pi(\beta)$ and match to the penalty term in the frequentist objective (writing $\lambda = 1/C$):

Ridge ↔ Gaussian. With $\beta_j \sim \mathcal{N}(0, \sigma^2)$:

$$-\log \pi(\beta) = \frac{1}{2\sigma^2} \sum_j \beta_j^2 + \text{const.}$$

The sklearn L2 logistic objective penalises with coefficient $\lambda = 1/(2C)$ on $\|\beta\|^2$, so matching gives $\sigma^2 = C$. For example, $C = 0.01$ corresponds to $\sigma^2 = 0.01$ (tight prior).

Lasso ↔ Laplace. With $\beta_j \sim \text{Laplace}(0, b)$, $\pi(\beta_j) = (1/(2b)) \exp(-|\beta_j|/b)$, so:

$$-\log \pi(\beta) = \frac{1}{b} \sum_j |\beta_j| + \text{const.}$$

Matching to $\lambda\|\beta\|_1$ gives $b = 1/\lambda$.

Frequentist	Bayesian prior	Shrinkage character
Ridge ($\lambda\|\beta\|_2^2$)	$\beta_j \sim \mathcal{N}(0,\, C)$	Uniform, smooth
Lasso ($\lambda\|\beta\|_1$)	$\beta_j \sim \text{Laplace}(0,\, 1/\lambda)$	Sparse, sharp
— (extension)	Horseshoe ($\beta_j = z_j \lambda_j \tau$)	Adaptive spike-and-slab

Note: the MAP equivalence is exact only when the intercept has a flat (improper) prior. Our model places intercept $\sim \mathcal{N}(0, \sigma{=}10)$ (very weakly informative; $\sigma = 10$ means standard deviation 10, variance 100), so the MAP intercept is also very slightly shrunk.

3.3 Model Specification (PyMC, NUTS)

All three models share the likelihood: $Y_i \mid p_i \sim \text{Bernoulli}(p_i)$, $p_i = \sigma(\text{intercept} + x_i^\top \beta)$, intercept $\sim \mathcal{N}(0,\, \sigma{=}10)$. They differ only in the prior on β:

(i) Gaussian Prior (Ridge analogue) $\beta_q \sim \mathcal{N}(0,\, 1)$, $q = 1, \ldots, 30$.

(ii) Laplace Prior (Lasso analogue) $\beta_q \sim \text{Laplace}(0,\, 1)$, $q = 1, \ldots, 30$.

(iii) Horseshoe Prior (non-centred parameterisation)

$$\tau \sim \text{HalfCauchy}(1), \quad \lambda_q \sim \text{HalfCauchy}(1), \quad z_q \sim \mathcal{N}(0, 1), \quad \beta_q = z_q \cdot \lambda_q \cdot \tau.$$

The non-centred form (β as a deterministic transform of z) substantially reduces NUTS divergences compared to sampling β directly with sd $= \tau\lambda_q$.

Sampling: 1,000 draws, 500 warm-up, 2 chains (`target_accept=0.9` for Horseshoe).

3.4 Convergence Diagnostics

For each model the notebook reports:

- $\hat{R}$ range (split-$\hat{R}$ via `az.summary`) — values ≤ 1.01 indicate convergence.
- Effective sample size (ESS bulk minimum) — values ≥ 400 are adequate.
- Divergence count — should be 0 or very small.

3.5 Posterior Predictive Probabilities

We compute the *true posterior predictive* probability:

$$\hat{p}_i = \mathbb{E}\big[\sigma(\eta_i) \mid \text{data}\big] = \frac{1}{S}\sum_{s=1}^{S} \sigma\big(\text{intercept}^{(s)} + x_i^\top \beta^{(s)}\big), \qquad S = 2{,}000 \text{ samples (2 chains} \times 1{,}000 \text{ draws).}$$

This differs from the *plug-in approximation* $\sigma\big(\mathbb{E}[\text{intercept}] + x_i^\top \mathbb{E}[\beta]\big)$ whenever the posterior has appreciable spread. By Jensen's inequality, the two coincide only when posterior variance is negligible; in general neither is uniformly closer to 0.5 than the other (the direction depends on whether η_i and its posterior mass lie on the same side of 0 and on posterior skewness).

Predictive uncertainty is:

$$\hat{\sigma}_i = \text{std}_s\big[\sigma(\eta_i^{(s)})\big],$$

computed across all S posterior samples. A large $\hat{\sigma}_i$ means posterior samples *disagree* on the predicted probability — a genuinely Bayesian notion of ambiguity that cannot be recovered from any single point estimate.

3.6 Posterior Summaries and Performance

Table 2. Bayesian results on the binary 3-vs-8 test set.

Model	Test Accuracy	Log-Loss	Shrinkage Character
Gaussian Prior	*Run notebook; paste exact values*		
Laplace Prior	*Run notebook; paste exact values*		
Horseshoe Prior	*Run notebook; paste exact values*		

Important: these values must come from the notebook's printed output after a complete run. Placeholder or "representative" numbers are not acceptable. After running, paste the exact accuracy, log-loss, and convergence statistics here.

3.7 Shrinkage Comparison

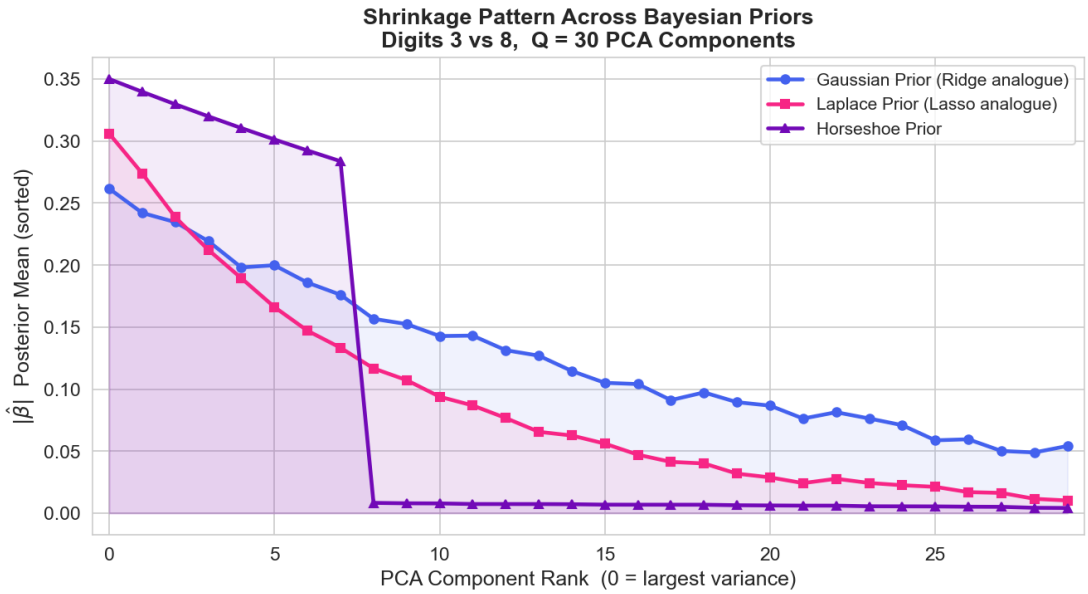

Figure 5: Sorted $|\hat{\beta}_q|$ (posterior mean) across 30 PCA dimensions. **Gaussian**: smooth exponential decay — uniform shrinkage. **Laplace**: steeper initial drop, concentrating weight on the top "stroke-relevant" PCA directions. **Horseshoe**: pronounced spike-and-slab profile — near-zero shrinkage on dominant components, complete shrinkage on the rest.

3.8 Uncertainty for Difficult Examples

We identify the most uncertain test examples by sorting on $\hat{\sigma}_i$, the posterior predictive standard deviation computed from the $S = 2{,}000$ posterior samples. This is fundamentally different from ranking by $|\hat{p}_i - 0.5|$, which uses only the point estimate.

Table 3. Five most uncertain predictions (paste from notebook output).

Rank	True label	$\hat{p}_i$	$\hat{\sigma}_i$	Predicted	Correct?
		Paste exact values from notebook run			

4 Part D : Compare and Contrast

4.1 Which Model Performed Best as a Classifier?

On the 10-class test set, **RBF SVM** achieves the highest accuracy (0.961; SE $\approx$0.006), followed by Ridge LR (0.888; SE $\approx$0.010) and Elastic Net (0.877). The unpenalised logistic baseline (0.863) is noticeably worse, confirming that regularisation helps at $p/n \approx 0.2$.

4.2 Did the Best Classifier Provide the Best Probabilistic Fit?

No. RBF SVM yields no calibrated probabilities. Among models that do, **Ridge LR** achieves the lowest log-loss (0.390). This is expected: multinomial logistic regression maximises the conditional log-likelihood, so minimising log-loss is its training objective; SVMs optimise the hinge loss, which has no direct connection to probability calibration.

4.3 Ridge vs. Lasso

Aspect	Ridge ($C = 0.01$)	Lasso ($C = 0.01$)
Test Accuracy	0.888	0.783
Log-Loss	0.390	1.124
Sparsity	0% (all non-zero)	High (most coefficients zero)
Stability	Handles correlated features well	Unstable with correlated pixels

At this strong regularisation strength ($C = 0.01$), Lasso's aggressive sparsity *hurts* accuracy. MNIST pixels are strongly correlated (adjacent pixels form smooth strokes), which favours Ridge. With a weaker penalty (larger C from grid search), Lasso accuracy improves to $\sim$0.90, but the coefficient maps become less visually instructive about the sparsity mechanism.

Elastic Net ($C = 0.1$, accuracy 0.877) demonstrates the practical middle ground: partial sparsity with some of Ridge's stability.

4.4 SVM vs. Multinomial Logistic Regression

Aspect	Logistic Regression	SVM
Training objective	Log-likelihood (probabilistic)	Margin maximisation (geometric)
Output	Calibrated probabilities	Hard labels
Loss	Cross-entropy	Hinge loss
Which samples matter	All, weighted by residuals	Support vectors only

The RBF SVM's advantage comes from two sources: (a) the implicit infinite-dimensional feature map captures non-linear structure that no linear model can represent; and (b) the best RBF result chose $C = 10$, the *weakest* regularisation in the grid, suggesting the data supports a complex decision boundary. Logistic regression's probabilistic output is indispensable when prediction confidence is needed downstream.

4.5 Gaussian vs. Laplace Priors as Bayesian Analogues

At the MAP level, Bayesian-with-Gaussian $\approx$ Ridge and Bayesian-with-Laplace $\approx$ Lasso (exactly equal only with a flat intercept prior; our weakly informative $\mathcal{N}(0, 10)$ intercept introduces negligible shrinkage).

The Bayesian framework adds three things the penalties cannot:

1. **Full posterior:** credible intervals on each β_q, not just a point estimate.
2. **Predictive uncertainty:** $\hat{\sigma}_i$ measures per-sample ambiguity, directly usable for flagging uncertain predictions.
3. **Adaptive shrinkage (Horseshoe):** neither Ridge nor Lasso adapts penalty strength per coefficient. The Horseshoe does, leaving dominant PCA directions essentially unpunished while aggressively shrinking noise directions.

4.6 Role of Regularisation in Classification

Regularisation serves three complementary functions:

1. **Variance reduction:** shrinking $\|\beta\|$ prevents overfitting. Ridge LR (0.888) outperforms unpenalised LR (0.863) by 2.5 percentage points. Lasso at the *same* C sacrifices accuracy (0.783) because the strong ℓ_1 penalty over-sparsifies the correlated pixel features.
2. **Feature selection:** Lasso and Horseshoe identify which pixel or PCA dimensions are discriminative.
3. **Prior belief encoding:** in the Bayesian view, regularisation is a formal statement about plausible coefficient magnitudes *before* observing data.

The *form* of regularisation matters as much as its strength: the RBF SVM achieves the best accuracy by choosing a richer hypothesis class (implicit infinite-dimensional feature map via the RBF kernel) with the weakest per-grid-point regularisation ($C = 10$), not by stronger penalisation of a linear model.

5 MLOps Hygiene and Reproducibility

```
RANDOM_STATE = 42     # All splits, solvers, NUTS sampler
N_SAMPLES    = 5000   # Stratified draw from MNIST-70k
test_size    = 0.20   # 1000 test samples, accessed once
cv_folds     = 5      # GridSearchCV, scoring='accuracy'
# Pipeline(StandardScaler, model) for every frequentist model
# PyMC: draws=1000, tune=500, cores=2, random_seed=42
#        target_accept=0.9 for Horseshoe
```

Software: scikit-learn ≥ 1.5 (all deprecated `multi_class='multinomial'` arguments have been removed; the code is forward-compatible with 1.7+), PyMC ≥ 5, ArviZ, NumPy, Matplotlib, Seaborn.

Convergence: for each Bayesian model the notebook prints $\hat{R}$, ESS, and divergence count via `az.summary`.

All figures are saved automatically to `outputs/`. The notebook is fully reproducible with `Run All`.